Template

Billy Wang

2026 年 4 月 30 日

目录

1 写在前面 **3**
1.1 基础模版 . . . 3
1.2 vimrc . . . 3

2 数据结构 **4**
2.1 zkw 线段树 . . . 4
2.2 珂朵莉树 . . . 5
2.3 FHQ-Treap . . . 5
2.4 并查集 . . . 6
2.5 ST 表 . . . 7
2.6 树状数组 . . . 8
2.7 线段树 . . . 8
2.7.1 我一般写的 . . . 8
2.7.2 一款较通用的 . . . 10

3 数学 **12**
3.1 快速幂 . . . 12
3.2 高斯消元 . . . 12
3.3 筛法 . . . 13
3.3.1 埃式筛 . . . 13
3.3.2 线性筛 . . . 13
3.4 类欧几里得 . . . 14
3.4.1 经典问题/我自己写的 . . . 14
3.4.2 洛谷模版题/OI-Wiki . . . 15
3.5 递推组合数 . . . 16
3.6 矩阵快速幂 . . . 16
3.7 扩展欧几里得 . . . 17
3.8 快速莫比乌斯/沃尔什变换 . . . 18
3.9 子集卷积 . . . 20

4 图论 **21**
4.1 倍增 21
4.2 网络流 22
4.2.1 最大流 22
4.2.2 费用流 24
4.3 二分图最大匹配 26
4.4 Tarjan 强连通分量缩点 27
4.5 树直径 27
4.6 树重心 28
4.7 树链剖分 29
4.8 最短路 31
4.8.1 Floyd（最小环） 31
4.8.2 Spfa（判负环） 31
4.8.3 Dijkstra 32
4.9 拓扑排序 33
4.10 最小生成树 33
4.11 欧拉路径/回路 34

5 字符串 **34**
5.1 KMP 34
5.2 Trie 树 35
5.3 Manacher 35
5.4 回文自动机 37
5.5 字符串哈希 38
5.6 后缀数组 39

6 STL **40**
6.1 算法库 40

1 写在前面

1.1 基础模版

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define REP( i, a, b ) for ( int i = ( a ); i <= ( b ); ++i )
#define REPd( i, a, b ) for ( int i = ( a ); i >= ( b ); --i )
inline ll rd() { /* {{{ */
  ll r = 0, k = 1;
  char c;
  while ( !isdigit( c = getchar() ) )
    if ( c == '-' )
      k = -k;
  while ( isdigit( c ) )
    r = r * 10 + c - '0', c = getchar();
  return r * k;
}
inline void prt( ll x ) {
  printf( "%lld ", x );
} /* }}} */

int main() {

  return 0;
}
```

1.2 vimrc

```
syntax on
set ts=2 sw=2 sts=2
set et ai cin nu sm ru cul
set mouse=a title wim=list nowrap ls=2
set ml sc
set mls=5
set fdm=marker
colo desert

im <C-S> <esc>:w<CR>
```

```
nn <C-S> :w<CR>
nn <space> za

" 使用 Windows 环境时可能需要
"   set shell=powershell
"   set backspace=indent,eol,start
"   set nocompatible
" set sh=powershell bs=indent,eol,start nocp
"
" 如果 <C-S> 导致 vim 卡死
" 在 .bashrc 里加一句 stty -ixon
```

2 数据结构

2.1 zkw 线段树

单点修区间查

```
ll s[N << 2], a[N];
int M;

ll f( ll x, ll y ) {
  return x + y; // 改这
}

void build() {
  for ( M = 1; M <= n + 1; M <<= 1 ) {}
  REP( i, 1, n ) s[i + M] = a[i];
  REPd( i, M - 1, 1 ) s[i] = f( s[2 * i], s[2 * i + 1] );
}

ll qrange( int l, int r, ll init ) { // 根据 f 传 init
  ll res = init;
  for ( l = l + M - 1, r = r + M + 1; l ^ r ^ 1; l >>= 1, r >>= 1 )
      {
    if ( ~l & 1 )
      res = f( res, s[l ^ 1] );
    if ( r & 1 )
      res = f( res, s[r ^ 1] );
  }
  return res;
```

```
}

void edit( int x, ll v ) {
  for ( s[x += M] = v, x >>= 1; x; x >>= 1 ) {
    s[x] = f( s[2 * x], s[2 * x + 1] );
  }
}

ll qpoint( int x ) {
  return s[x + M];
}
```

2.2 珂朵莉树

```
struct node {
  int l, r;
  mutable int v;
  bool operator<( const node& rhs ) const { return l < rhs.l; }
};

set<node> odt;
typedef set<node>::iterator iter;

iter split( ll p ) {
  iter tmp = odt.lower_bound( (node) { p, 0, 0 } );
  if ( tmp != odt.end() && tmp->l == p )
    return tmp;
  --tmp;
  int tl = tmp->l, tr = tmp->r, tv = tmp->v;
  odt.erase( tmp );
  odt.insert( (node) { tl, p - 1, tv } );
  return odt.insert( (node) { p, tr, tv } ).first;
}

// 修改和查询注意 split 顺序
// iter itr=split(r+1), itl=split(l);
```

2.3 FHQ-Treap

以模版文艺平衡树为例

```
int n, m, clk, rt;
struct node {
  int key, val, sz, tag, ls, rs;
} t[N];
int newnode( int k ) {
  return t[++clk] = (node) { k, rand(), 1, 0 }, clk;
}
void down( int o ) {
  if ( t[o].tag ) {
    t[t[o].ls].tag = 1 - t[t[o].ls].tag;
    t[t[o].rs].tag = 1 - t[t[o].rs].tag;
    swap( t[t[o].ls].ls, t[t[o].ls].rs );
    swap( t[t[o].rs].ls, t[t[o].rs].rs );
    t[o].tag = 0;
  }
}
void up( int o ) {
  t[o].sz = t[t[o].ls].sz + t[t[o].rs].sz + 1;
}
void split( int o, int x, int& L, int& R ) {
  if ( o == 0 )
    return L = R = 0, void();
  down( o );
  if ( t[t[o].ls].sz + 1 >= x )
    R = o, split( t[o].ls, x, L, t[o].ls );
  else
    L = o, split( t[o].rs, x - t[t[o].ls].sz - 1, t[o].rs, R );
  up( o );
}
int merge( int L, int R ) {
  if ( L == 0 || R == 0 )
    return L + R;
  if ( t[L].val > t[R].val )
    return down( L ), t[L].rs = merge( t[L].rs, R ), up( L ), L;
  else
    return down( R ), t[R].ls = merge( L, t[R].ls ), up( R ), R;
}
```

2.4 并查集

```
ll n, fa[N];
void init() {
  iota( fa + 1, fa + n + 1, 1 );
}

int find( int x ) {
  if ( x == fa[x] )
    return x;
  return fa[x] = find( fa[x] );
}

void merge( int x, int y ) {
  x = find( x ), y = find( y );
  if ( x != y )
    fa[x] = y;
}
```

2.5 ST 表

```
ll n, a[N], st[N][SP + 10], to[N][SP + 10], l2g[N];
ll op( ll x, ll y ) {
  return max( x, y );
}

void init() {
  l2g[1] = 0, to[n + 1][0] = n + 1;
  REP( i, 2, n ) l2g[i] = l2g[i - 1] + !( i & ( i - 1 ) );
  REP( i, 1, n ) st[i][0] = a[i], to[i][0] = i + 1;
  REP( i, 1, SP ) {
    REP( j, 1, n ) {
      to[j][i] = to[to[j][i - 1]][i - 1];
      st[j][i] = op( st[j][i - 1], st[to[j][i - 1]][i - 1] );
    }
  }
}

ll query( ll l, ll r ) { // [l, r] 闭区间
  ++r;
  ll d = l2g[r - l];
  return op( st[l][d], st[r - ( 1 << d )][d] );
```

```
}
```

2.6 树状数组

```
ll n, fwt[N];
ll prod( ll x, ll d ) {
  return x + d;
}
ll op( ll x, ll y ) {
  return x + y;
}

void edit( int x, ll d ) {
  for ( ; x <= n; x += x & -x )
    fwt[x] = prod( fwt[x], d );
}

ll query( int x ) {
  assert( 1 <= x && x <= n );
  ll res = fwt[x];
  x -= x & -x;
  // 这种写法不用考虑最大或最小值的初值问题
  for ( ; x; x -= x & -x )
    res = op( res, fwt[x] );
  return res;
}
```

2.7 线段树

2.7.1 我一般写的

```
ll n, a[N];
struct seg {
  int l, r;
  ll sum, tag;
} s[N << 2];
#define lson ( o * 2 )
#define rson ( o * 2 + 1 )

void down( int o ) {
```

```
  if ( s[o].tag ) {
    s[lson].sum += s[o].tag * ( s[lson].r - s[lson].l + 1 ),
      s[lson].tag += s[o].tag;
    s[rson].sum += s[o].tag * ( s[rson].r - s[rson].l + 1 ),
      s[rson].tag += s[o].tag;
    s[o].tag = 0;
  }
}

void build( int o, int l, int r ) {
  s[o].l = l, s[o].r = r;
  if ( l == r )
    return s[o].sum = a[l], void();
  int mid = ( l + r ) / 2;
  build( lson, l, mid );
  build( rson, mid + 1, r );
  s[o].sum = s[lson].sum + s[rson].sum;
}

void add( int o, int l, int r, ll d ) {
  if ( l <= s[o].l && s[o].r <= r )
    return s[o].sum += d * ( s[o].r - s[o].l + 1 ), s[o].tag += d,
        void();
  down( o );
  int mid = ( s[o].l + s[o].r ) / 2;
  if ( l <= mid )
    add( lson, l, r, d );
  if ( r > mid )
    add( rson, l, r, d );
  s[o].sum = s[lson].sum + s[rson].sum;
}

ll sum( int o, int l, int r ) {
  if ( l <= s[o].l && s[o].r <= r )
    return s[o].sum;
  down( o );
  int mid = ( s[o].l + s[o].r ) / 2;
  ll res = 0;
  if ( l <= mid )
    res += sum( lson, l, r );
```

```
  if ( r > mid )
    res += sum( rson, l, r );
  return res;
}
```

2.7.2 一款较通用的

```
ll n, a[N];

namespace sg {

struct Q { // 存修改，存 tag
  ll tag;
  Q( ll tag = 0 ) : tag( tag ) {} // 注意初始值
  void operator+=( const Q& q ) { tag += q.tag; }
  // tag 的叠加
};
struct P { // 存记录维护的答案
  ll sum;
  P( ll sum = 0 ) : sum( sum ) {} // 注意初始值
  void push( const Q& q, int l, int r ) { sum += q.tag * ( r - l +
      1 ); }
  // 区间从操作更新答案
  void init( int x ) { sum = a[x]; } // 建树中的赋值
};
P operator&( const P& a, const P& b ) { // 子区间合并答案
  return P( a.sum + b.sum );
}
// 基本上只需要修改上面的部分
P p[N << 2];
Q q[N << 2];
#define lson o * 2, l, ( l + r ) / 2
#define rson o * 2 + 1, ( l + r ) / 2 + 1, r
void up( int o, int l, int r ) {
  if ( l < r )
    p[o] = p[o * 2] & p[o * 2 + 1];
}
void push( const Q& v, int o, int l, int r ) {
  q[o] += v;
  p[o].push( v, l, r );
```

```
}
void down( int o, int l, int r ) {
  push( q[o], lson );
  push( q[o], rson );
  q[o] = Q();
}
void build( int o = 1, int l = 1, int r = n ) {
  if ( l == r )
    p[o].init( l );
  else {
    build( lson );
    build( rson );
  }
  up( o, l, r );
}
P query( int ql, int qr, int o = 1, int l = 1, int r = n ) {
  if ( ql > r || l > qr )
    return P();
  if ( ql <= l && r <= qr )
    return p[o];
  down( o, l, r );
  return query( ql, qr, lson ) & query( ql, qr, rson );
}
void update( int ql, int qr, const Q& v, int o = 1, int l = 1, int
    r = n ) {
  if ( ql > r || l > qr )
    return;
  if ( ql <= l && r <= qr ) {
    push( v, o, l, r );
    return;
  }
  down( o, l, r );
  update( ql, qr, v, lson );
  update( ql, qr, v, rson );
  up( o, l, r );
}

}
using namespace sg;
```

3 数学

3.1 快速幂

```
const ll MOD = 998244353; // 改模数

ll qpow( ll a, ll x ) {
  ll res = 1;
  a %= MOD;
  while ( x ) {
    if ( x & 1 )
      res = res * a % MOD;
    a = a * a % MOD, x >>= 1;
  }
  return res;
}

ll inv( ll x ) {
  return qpow( x, MOD - 2 );
} // 模数为质数时
```

3.2 高斯消元

```
const int N = 110;
ll n;
double a[N][N], b[N];
void work() {
  n = rd();
  REP( i, 1, n ) {
    REP( j, 1, n ) a[i][j] = rd();
    b[i] = rd();
  }
  REP( i, 1, n ) {
    int t = i;
    REP( j, i + 1, n )
    if ( abs( a[j][i] ) > 1e-7
         && ( abs( a[t][i] ) > abs( a[j][i] ) || abs( a[t][i] ) < 1
            e-7 ) )
      t = j;
    REP( j, i, n ) swap( a[t][j], a[i][j] );
```

```
    if ( abs( a[i][i] ) < 1e-7 ) {
      puts( "No Solution" );
      return 0;
    }
    swap( b[t], b[i] );
    double e = a[i][i];
    REP( j, i, n ) a[i][j] /= e;
    b[i] /= e;
    REP( j, i + 1, n ) {
      double d = a[j][i];
      REP( k, i, n ) a[j][k] -= d * a[i][k];
      b[j] -= d * b[i];
    }
  }
  REPd( i, n, 1 ) REP( j, 1, i - 1 ) b[j] -= a[j][i] * b[i], a[j][i
      ] = 0;
  // REP(i, 1, n) printf("%.2f\n", b[i]);
  // b[1...n] 保存 Ax=b 的解
}
```

3.3 筛法

3.3.1 埃式筛

```
bitset<N> b;
ll n;
vector<ll> prime;
void erato() {
  REP( i, 2, n ) if ( !b[i] ) {
    prime.push_back( i );
    for ( int j = i + i; j <= n; j += i )
      b[j] = 1;
  }
}
```

3.3.2 线性筛

```
bitset<N> b;
ll n, phi[N];
vector<ll> prime;
void euler() {
```

```
  REP( i, 2, n ) {
    if ( !b[i] ) {
      prime.push_back( i );
      phi[i] = i - 1;
    }
    for ( int p : prime ) {
      if ( p * i > n )
        break;
      b[p * i] = 1;
      if ( i % p == 0 ) {
        phi[p * i] = phi[i] * p;
        break;
      }
      phi[p * i] = phi[i] * phi[p];
    }
  }
}
```

3.4 类欧几里得

3.4.1 经典问题/我自己写的

```
ll floordiv( ll x, ll y ) {
  assert( y != 0 );
  if ( y < 0 )
    x = -x;
  return x >= 0 ? ( x / y ) : ( ( x - y + 1 ) / y );
}

ll mod( ll x, ll y ) {
  return ( x % y + y ) % y;
}

ll f( ll a, ll b, ll c, ll n ) {
  if ( c < 0 )
    a = -a, b = -b, c = -c;
  assert( c != 0 );
  ll res = ( floordiv( a, c ) % MOD + MOD ) % MOD
             * ( ( n * ( n + 1 ) / 2 ) % MOD ) % MOD
           + ( floordiv( b, c ) % MOD + MOD ) % MOD * ( n + 1 ) %
              MOD;
```

```
  res %= MOD;
  a = mod( a, c ), b = mod( b, c );
  if ( !a )
    return res;
  res += ( n + 1 ) * ( floordiv( a * n + b, c ) % MOD ) % MOD
         - floordiv( b, a ) % MOD;
  res = ( res % MOD + MOD ) % MOD;
  res -= f( c, a - b - 1, a, floordiv( a * n + b, c ) );
  res = ( res % MOD + MOD ) % MOD;
  return res;
}
```

3.4.2 洛谷模版题/OI-Wiki

$f(a,b,c,n)=\sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor$
$g(a,b,c,n)=\sum_{i=0}^{n}i\lfloor\frac{ai+b}{c}\rfloor$
$h(a,b,c,n)=\sum_{i=0}^{n}\lfloor\frac{an+b}{c}\rfloor^2$

```
struct Data {
  int f, g, h;
  // f(a, b, c, n)=\sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor
  // g(a, b, c, n)=\sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor
  // h(a, b, c, n)=\sum_{i=0}^n \lfloor \frac{an+b}{c} \rfloor ^2
};

Data solve( long long a, long long b, long long c, long long n ) {
  constexpr long long M = 998244353;
  constexpr long long i2 = ( M + 1 ) / 2;
  constexpr long long i6 = ( M + 1 ) / 6;
  long long n2 = ( n + 1 ) * n % M * i2 % M;
  long long n3 = ( 2 * n + 1 ) * ( n + 1 ) % M * n % M * i6 % M;
  Data res = { 0, 0, 0 };
  if ( a >= c || b >= c ) {
    auto tmp = solve( a % c, b % c, c, n );
    long long aa = a / c, bb = b / c;
    res.f = ( tmp.f + aa * n2 + bb * ( n + 1 ) ) % M;
    res.g = ( tmp.g + aa * n3 + bb * n2 ) % M;
    res.h
      = ( tmp.h + 2 * bb * tmp.f % M + 2 * aa * tmp.g % M + aa * aa
          % M * n3 % M
          + bb * bb % M * ( n + 1 ) % M + 2 * aa * bb % M * n2 % M
```

```
            )
        % M;
    return res;
  }
  long long m = ( a * n + b ) / c;
  if ( !m )
    return res;
  auto tmp = solve( c, c - b - 1, a, m - 1 );
  res.f = ( m * n - tmp.f + M ) % M;
  res.g = ( m * n2 + ( M - tmp.f ) * i2 + ( M - tmp.h ) * i2 ) % M;
  res.h = ( n * m % M * m - tmp.f - tmp.g * 2 + 3 * M ) % M;
  return res;
}
```

3.5 递推组合数

```
ll C[N][N];
void calcC( int n ) {
  C[0][0] = 1;
  REP( i, 1, n ) {
    C[i][0] = C[i][i] = 1;
    REP( j, 1, i - 1 ) C[i][j] = ( C[i - 1][j] + C[i - 1][j - 1] )
        % MOD;
  }
}
```

3.6 矩阵快速幂

```
struct mat {
  ll n, m;
  vector<vector<ll>> val; // 注意下标从 0 开始
  mat( ll _n, ll _m ) : n( _n ), m( _m ) {
    val.resize( _n );
    REP( i, 0, _n - 1 ) val[i].resize( _m, 0 );
  }
  mat( ll _n, ll _m, vector<vector<ll>> _val )
    : n( _n ), m( _m ), val( _val ) {}
};

mat mul( const mat& x, const mat& y, ll mod ) {
```

```
  assert( x.m == y.n );
  mat res( x.n, y.m );
  REP( i, 0, res.n - 1 ) REP( j, 0, res.m - 1 ) {
    REP( k, 0, x.m - 1 ) {
      res.val[i][j] = ( res.val[i][j] + x.val[i][k] * y.val[k][j] %
          mod ) % mod;
    }
  }
  return res;
}

mat qpow( mat a, ll x, ll mod ) {
  assert( a.n == a.m );
  mat res( a.n, a.n );
  REP( i, 0, a.n - 1 ) res.val[i][i] = 1;
  REP( i, 0, a.n - 1 ) REP( j, 0, a.m - 1 ) a.val[i][j] %= mod;
  while ( x ) {
    if ( x & 1 )
      res = mul( res, a, mod );
    a = mul( a, a, mod ), x >>= 1;
  }
  return res;
}
```

3.7 扩展欧几里得

```
ll exgcd( ll a, ll b, ll& x, ll& y ) {
  if ( b == 0 ) {
    x = 1, y = 0;
    return a;
  }
  ll xx = 0, yy = 0;
  ll res = exgcd( b, a % b, xx, yy );
  y = xx - ( a / b ) * yy, x = yy;
  return res;
}

ll inv( ll x, ll mod ) {
  ll xx, yy;
  ll d = exgcd( x, mod, xx, yy );
```

```
  assert( d == 1 );
  return ( xx % mod + mod ) % mod;
}
```

3.8 快速莫比乌斯/沃尔什变换

```
int n, m;
ll a[N], b[N], c[N];
ll inv2;

void solor() {
  for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < m; ++j ) {
      if ( ( j >> i ) & 1 )
        continue;
      ( a[j + ( 1 << i )] += a[j] ) %= MOD;
      ( b[j + ( 1 << i )] += b[j] ) %= MOD;
    }
  }
  for ( int j = 0; j < m; ++j )
    c[j] = ( a[j] * b[j] ) % MOD;
  for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < m; ++j ) {
      if ( ( j >> i ) & 1 )
        continue;
      a[j + ( 1 << i )] = ( a[j + ( 1 << i )] - a[j] + MOD ) % MOD;
      b[j + ( 1 << i )] = ( b[j + ( 1 << i )] - b[j] + MOD ) % MOD;
      c[j + ( 1 << i )] = ( c[j + ( 1 << i )] - c[j] + MOD ) % MOD;
    }
  }
  for ( int j = 0; j < m; ++j )
    prt( c[j] );
  puts( "" );
}

void soland() {
  for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < m; ++j ) {
      if ( ( j >> i ) & 1 )
        continue;
```

```
      ( a[j] += a[j + ( 1 << i )] ) %= MOD;
      ( b[j] += b[j + ( 1 << i )] ) %= MOD;
    }
  }
  for ( int j = 0; j < m; ++j )
    c[j] = ( a[j] * b[j] ) % MOD;
  for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < m; ++j ) {
      if ( ( j >> i ) & 1 )
        continue;
      a[j] = ( a[j] - a[j + ( 1 << i )] + MOD ) % MOD;
      b[j] = ( b[j] - b[j + ( 1 << i )] + MOD ) % MOD;
      c[j] = ( c[j] - c[j + ( 1 << i )] + MOD ) % MOD;
    }
  }
  for ( int j = 0; j < m; ++j )
    prt( c[j] );
  puts( "" );
}

void solxor() {
  for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < m; ++j ) {
      if ( ( j >> i ) & 1 )
        continue;
      ( a[j] += a[j + ( 1 << i )] ) %= MOD;
      a[j + ( 1 << i )] = ( a[j] - 2 * a[j + ( 1 << i )] + 2 * MOD
          ) % MOD;
      ( b[j] += b[j + ( 1 << i )] ) %= MOD;
      b[j + ( 1 << i )] = ( b[j] - 2 * b[j + ( 1 << i )] + 2 * MOD
          ) % MOD;
    }
  }
  for ( int j = 0; j < m; ++j )
    c[j] = ( a[j] * b[j] ) % MOD;
  for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < m; ++j ) {
      if ( ( j >> i ) & 1 )
        continue;
      a[j] = ( ( a[j] + a[j + ( 1 << i )] ) * inv2 ) % MOD;
```

```
      a[j + ( 1 << i )] = ( a[j] - a[j + ( 1 << i )] + MOD ) % MOD;
      b[j] = ( ( b[j] + b[j + ( 1 << i )] ) * inv2 ) % MOD;
      b[j + ( 1 << i )] = ( b[j] - b[j + ( 1 << i )] + MOD ) % MOD;
      c[j] = ( ( c[j] + c[j + ( 1 << i )] ) * inv2 ) % MOD;
      c[j + ( 1 << i )] = ( c[j] - c[j + ( 1 << i )] + MOD ) % MOD;
    }
  }
  for ( int j = 0; j < m; ++j )
    prt( c[j] );
  puts( "" );
}
```

3.9 子集卷积

```
const ll MOD = 1e9 + 9;
const int N = 1050576, SP = 25;
int n, m;
int popcnt[N];
ll a[SP][N], b[SP][N], c[SP][N];
ll inv2;

void sol() {
  for ( int k = 0; k <= n; ++k ) {
    for ( int i = 0; i < n; ++i ) {
      for ( int j = 0; j < m; ++j ) {
        if ( ( j >> i ) & 1 )
          continue;
        ( a[k][j + ( 1 << i )] += a[k][j] ) %= MOD;
        ( b[k][j + ( 1 << i )] += b[k][j] ) %= MOD;
      }
    }
  }
  for ( int k = 0; k <= n; ++k ) {
    for ( int l = 0; l <= k; ++l ) {
      for ( int j = 0; j < m; ++j ) {
        c[k][j] = ( c[k][j] + a[l][j] * b[k - l][j] % MOD ) % MOD;
      }
    }
  }
  for ( int k = 0; k <= n; ++k ) {
```

```
    for ( int i = 0; i < n; ++i ) {
      for ( int j = 0; j < m; ++j ) {
        if ( ( j >> i ) & 1 )
          continue;
        c[k][j + ( 1 << i )] = ( c[k][j + ( 1 << i )] - c[k][j] +
            MOD ) % MOD;
      }
    }
  }
  for ( int j = 0; j < m; ++j )
    prt( c[popcnt[j]][j] );
  puts( "" );
}
```

4 图论

4.1 倍增

```
void dfs( int x, int fa ) {
  pa[x][0] = fa;
  dep[x] = dep[fa] + 1;
  REP( i, 1, SP ) pa[x][i] = pa[pa[x][i - 1]][i - 1];
  for ( int& v : g[x] )
    if ( v != fa ) {
      dfs( v, x );
    }
}

int lca( int x, int y ) {
  if ( dep[x] < dep[y] )
    swap( x, y );
  int t = dep[x] - dep[y];
  REP( i, 0, SP ) if ( t & ( 1 << i ) ) x = pa[x][i];
  REPd( i, SP - 1, -1 ) {
    int xx = pa[x][i], yy = pa[y][i];
    if ( xx != yy )
      x = xx, y = yy;
  }
  return x == y ? x : pa[x][0];
}
```

4.2 网络流

不是我写的，但是看着还好

其中 `ll` 是我改的，不敢保证有没有漏改，但是过了洛谷模版题

4.2.1 最大流

```
constexpr ll INF = LLONG_MAX / 2;

struct E {
  int to;
  ll cp;
  E( int to, ll cp ) : to( to ), cp( cp ) {}
};

struct Dinic {
  static const int M = 1E5 * 5;
  int m, s, t;
  vector<E> edges;
  vector<int> G[M];
  int d[M];
  int cur[M];

  void init( int n, int s, int t ) {
    this->s = s;
    this->t = t;
    for ( int i = 0; i <= n; i++ )
      G[i].clear();
    edges.clear();
    m = 0;
  }

  void addedge( int u, int v, ll cap ) {
    edges.emplace_back( v, cap );
    edges.emplace_back( u, 0 );
    G[u].push_back( m++ );
    G[v].push_back( m++ );
  }

  bool BFS() {
    memset( d, 0, sizeof d );
```

```
    queue<int> Q;
    Q.push( s );
    d[s] = 1;
    while ( !Q.empty() ) {
      int x = Q.front();
      Q.pop();
      for ( int& i : G[x] ) {
        E& e = edges[i];
        if ( !d[e.to] && e.cp > 0 ) {
          d[e.to] = d[x] + 1;
          Q.push( e.to );
        }
      }
    }
    return d[t];
  }

  ll DFS( int u, ll cp ) {
    if ( u == t || !cp )
      return cp;
    ll tmp = cp, f;
    for ( int& i = cur[u]; i < G[u].size(); i++ ) {
      E& e = edges[G[u][i]];
      if ( d[u] + 1 == d[e.to] ) {
        f = DFS( e.to, min( cp, e.cp ) );
        e.cp -= f;
        edges[G[u][i] ^ 1].cp += f;
        cp -= f;
        if ( !cp )
          break;
      }
    }
    return tmp - cp;
  }

  ll go() {
    ll flow = 0;
    while ( BFS() ) {
      memset( cur, 0, sizeof cur );
      flow += DFS( s, INF );
```

```
    }
    return flow;
  }
} DC;
```

4.2.2 费用流

```
constexpr ll INF = LLONG_MAX / 2;

struct E {
  int from, to;
  ll cp, v;
  E() {}
  E( int f, int t, ll cp, ll v ) : from( f ), to( t ), cp( cp ), v(
      v ) {}
};

struct MCMF {
  static const int M = 1E5 * 5;
  int n, m, s, t;
  vector<E> edges;
  vector<int> G[M];
  bool inq[M];
  ll d[M], a[M];
  int p[M];

  void init( int _n, int _s, int _t ) {
    n = _n;
    s = _s;
    t = _t;
    REP( i, 0, n + 1 ) G[i].clear();
    edges.clear();
    m = 0;
  }

  void addedge( int from, int to, ll cap, ll cost ) {
    edges.emplace_back( from, to, cap, cost );
    edges.emplace_back( to, from, 0, -cost );
    G[from].push_back( m++ );
    G[to].push_back( m++ );
```

```
}

bool BellmanFord( ll& flow, ll& cost ) {
  REP( i, 0, n + 1 ) d[i] = INF;
  memset( inq, 0, sizeof inq );
  d[s] = 0, a[s] = INF, inq[s] = true;
  queue<int> Q;
  Q.push( s );
  while ( !Q.empty() ) {
    int u = Q.front();
    Q.pop();
    inq[u] = false;
    for ( int& idx : G[u] ) {
      E& e = edges[idx];
      if ( e.cp && d[e.to] > d[u] + e.v ) {
        d[e.to] = d[u] + e.v;
        p[e.to] = idx;
        a[e.to] = min( a[u], e.cp );
        if ( !inq[e.to] ) {
          Q.push( e.to );
          inq[e.to] = true;
        }
      }
    }
  }
  if ( d[t] == INF )
    return false;
  flow += a[t];
  cost += a[t] * d[t];
  int u = t;
  while ( u != s ) {
    edges[p[u]].cp -= a[t];
    edges[p[u] ^ 1].cp += a[t];
    u = edges[p[u]].from;
  }
  return true;
}

pair<ll, ll> go() {
  ll flow = 0, cost = 0;
```

```
    while ( BellmanFord( flow, cost ) )
      ;
    return make_pair( flow, cost );
  }
} MM;
```

4.3 二分图最大匹配

ps. 建单向图（即只有左部指向右部的边）

```
struct MaxMatch {
  int n;
  vector<int> G[N];
  int vis[N], left[N], clk;

  void init( int n ) {
    this->n = n;
    REP( i, 0, n + 1 ) G[i].clear();
    memset( left, -1, sizeof left );
    memset( vis, -1, sizeof vis );
  }

  bool dfs( int u ) {
    for ( int v : G[u] )
      if ( vis[v] != clk ) {
        vis[v] = clk;
        if ( left[v] == -1 || dfs( left[v] ) ) {
          left[v] = u;
          return true;
        }
      }
    return false;
  }

  int match() {
    int ret = 0;
    for ( clk = 0; clk <= n; ++clk )
      if ( dfs( clk ) )
        ++ret;
    return ret;
  }
```

```
} MM;
```

4.4 Tarjan 强连通分量缩点

```
int low[N], dfn[N], clk, B, bl[N];
vector<int> bcc[N];
void init() {
  B = clk = 0;
  memset( dfn, 0, sizeof dfn );
}
void tarjan( int u ) {
  static int st[N], p;
  static bool in[N];
  dfn[u] = low[u] = ++clk;
  st[p++] = u;
  in[u] = true;
  for ( int& v : G[u] ) {
    if ( !dfn[v] ) {
      tarjan( v );
      low[u] = min( low[u], low[v] );
    } else if ( in[v] )
      low[u] = min( low[u], dfn[v] );
  }
  if ( dfn[u] == low[u] ) {
    ++B;
    while ( 1 ) {
      int x = st[--p];
      in[x] = false;
      bl[x] = B;
      bcc[B].push_back( x );
      if ( x == u )
        break;
    }
  }
}
```

4.5 树直径

```
ll n, dep[N], mxdep[N];
vector<int> g[N];
```

```
vector<int> dmt;
void dfs1( int x, int fa ) {
  dep[x] = dep[fa] + 1;
  mxdep[x] = 1;
  for ( int u : g[x] )
    if ( u != fa ) {
      dfs1( u, x );
      mxdep[x] = max( mxdep[u] + 1, mxdep[x] );
    }
}

void dfs2( int x, int fa ) { // 找一条直径，如果只需要直径长度则不用
  dmt.push_back( x );
  int nxt = -1;
  for ( int u : g[x] )
    if ( u != fa ) {
      if ( nxt == -1 || mxdep[u] > mxdep[nxt] )
        nxt = u;
    }
  if ( nxt != -1 )
    dfs2( nxt, x );
}

void diameter() {
  dep[0] = 0;
  dfs1( 1, 0 );
  int rt = max_element( dep + 1, dep + n + 1 ) - dep;
  dfs1( rt, 0 );
  dfs2( rt, 0 );
}
```

4.6 树重心

```
ll n, sz[N], mxsz[N], G;
vector<int> g[N];
void dfs( int x, int fa ) {
  sz[x] = 1, mxsz[x] = 0;
  for ( int u : g[x] )
    if ( u != fa ) {
```

```
      dfs( u, x );
      sz[x] += sz[u];
      mxsz[x] = max( sz[u], mxsz[x] );
    }
  mxsz[x] = max( mxsz[x], n - sz[x] );
  if ( G == -1 || ( mxsz[x] < mxsz[G] || ( mxsz[x] == mxsz[G] && x < G ) ) )
    G = x;
}
```

4.7 树链剖分

```
ll fa[N], dep[N], son[N], sz[N], top[N], idx[N], clk, ridx[N], out[N];
vector<int> g[N];
void predfs( int x, int f ) { // 第一次 dfs 计算父亲/深度/大小/重儿子
  fa[x] = f, dep[x] = dep[f] + 1, sz[x] = 1;
  for ( auto v : g[x] )
    if ( v != f ) {
      predfs( v, x );
      if ( sz[v] > sz[son[x]] )
        son[x] = v;
      sz[x] += sz[v];
    }
}

void dfs( int x, int tp ) { // 第二次 dfs
  // top 链的顶端
  // idx 节点在 dfs 序中的编号
  // ridx dfs 序中编号对应的节点
  // out 以该节点为根的子树的编号的最大值
  // 即 dfs 序中 [idx, out] 表示以它为根的子树
  // [idx[top[x]], idx[x]] 表示这条链的顶端到该节点的一段链
  top[x] = tp, idx[x] = ++clk, ridx[clk] = x;
  if ( son[x] != 0 )
    dfs( son[x], tp );
  for ( auto v : g[x] )
    if ( v != fa[x] && v != son[x] )
      top[v] = v, dfs( v, v );
```

```
  out[x] = clk;
}

void addpath( int u, int v, ll d ) {
  int uu = top[u], vv = top[v];
  while ( uu != vv ) {
    if ( dep[uu] < dep[vv] )
      swap( u, v ), swap( uu, vv );
    add( 1, idx[uu], idx[u], d );
    u = fa[uu], uu = top[u];
  }
  if ( dep[u] < dep[v] )
    swap( u, v );
  add( 1, idx[v], idx[u], d );
}

void addtree( int x, ll d ) {
  add( 1, idx[x], out[x], d );
}

ll sumpath( int u, int v ) {
  ll res = 0;
  int uu = top[u], vv = top[v];
  while ( uu != vv ) {
    if ( dep[uu] < dep[vv] )
      swap( u, v ), swap( uu, vv );
    res = ( res + sum( 1, idx[uu], idx[u] ) ) % P;
    u = fa[uu], uu = top[u];
  }
  if ( dep[u] < dep[v] )
    swap( u, v );
  res = ( res + sum( 1, idx[v], idx[u] ) ) % P;
  return res;
}

ll sumtree( int x ) {
  return sum( 1, idx[x], out[x] );
}
```

4.8 最短路

4.8.1 Floyd（最小环）

```
ll n, m, mincycle;
ll g[N][N], dis[N][N];
void floyd() {
  // 如果 g[i][j] 之间没边则存 inf
  // 注意 inf 的三倍不能爆 long long
  mincycle = LLONG_MAX / 4;
  REP( i, 1, n ) REP( j, 1, n ) dis[i][j] = g[i][j];
  REP( k, 1, n ) {
    REP( i, 1, k - 1 )
    REP( j, i + 1, k - 1 ) {
      mincycle = min( mincycle, dis[i][j] + g[j][k] + g[k][i] );
    }
    REP( i, 1, n )
    REP( j, 1, n )
    dis[i][j] = min( dis[i][j], dis[i][k] + dis[k][j] );
  }
}
```

4.8.2 Spfa（判负环）

```
ll n, m, dis[N], inq[N], cnt[N];
vector<pair<int, ll>> g[N];
bool spfa( int s ) {
  // 如果有负环则 return true
  queue<int> q;
  fill_n( dis + 1, n, LLONG_MAX / 2 );
  fill_n( inq + 1, n, 0 );
  fill_n( cnt + 1, n, 0 );
  dis[s] = 0, inq[s] = cnt[s] = 1;
  q.push( s );
  while ( !q.empty() ) {
    int u = q.front();
    q.pop();
    inq[u] = 0;
    for ( auto [v, w] : g[u] ) {
      if ( dis[v] > dis[u] + w ) {
        dis[v] = dis[u] + w;
```

```
        cnt[v] = cnt[u] + 1;
        if ( cnt[v] > n )
          return true;
        if ( !inq[v] ) {
          inq[v] = 1, q.push( v );
        }
      }
    }
  }
  return false;
}
```

4.8.3 Dijkstra

```
ll n, m, dis[N], vis[N];
vector<pair<int, ll>> g[N];
void dijkstra( int s ) {
  fill_n( dis + 1, n, LLONG_MAX / 2 );
  fill_n( vis + 1, n, 0 );
  dis[s] = 0;
  priority_queue<pair<ll, int>> q;
  q.push( make_pair( 0, s ) );
  while ( !q.empty() ) {
    int u = q.top().second;
    q.pop();
    if ( vis[u] )
      continue;
    vis[u] = 1;
    for ( auto [v, w] : g[u] ) {
      if ( vis[v] )
        continue;
      if ( dis[v] > dis[u] + w ) {
        dis[v] = dis[u] + w;
        q.push( make_pair( -dis[v], v ) );
      }
    }
  }
}
```

4.9 拓扑排序

```
ll n, in[N];
vector<int> g[N];
vector<int> res;
void topo() {
  // 不要忘记在建图时记录入度
  queue<int> q;
  REP( i, 1, n )
  if ( in[i] == 0 )
    q.push( i );
  while ( !q.empty() ) {
    int u = q.front();
    q.pop();
    res.push_back( u );
    for ( int v : g[u] ) {
      --in[v];
      if ( in[v] == 0 )
        q.push( v );
    }
  }
}
```

4.10 最小生成树

Kruskal 算法

```
ll n, m, fa[N], weight_sum;
pair<ll, pair<int, int>> edge[M];
int find( int x ) {
  if ( fa[x] == x )
    return x;
  return fa[x] = find( fa[x] );
}

void kruskal() {
  weight_sum = 0;
  sort( edge + 1, edge + m + 1 );
  iota( fa + 1, fa + n + 1, 1 );
  REP( i, 1, m ) {
    auto [u, v] = edge[i].second;
```

```
    u = find( u ), v = find( v );
    if ( u == v )
      continue;
    fa[u] = v;
    weight_sum += edge[i].first;
  }
}
```

4.11 欧拉路径/回路

```
ll n, m, cnt[N];
vector<int> g[N];
vector<int> path;
void dfs( int x ) {
  while ( cnt[x] < g[x].size() ) {
    int u = g[x][cnt[x]];
    ++cnt[x];
    dfs( u );
  }
  path.push_back( x );
  // 最后要 reverse(path.begin(), path.end());
}
```

5 字符串

5.1 KMP

```
char s1[N];
int n, pi[N];
void solve() {
  // 字符串存于 s1[0, n)
  pi[0] = 0;
  REP( i, 1, n - 1 ) {
    int j = pi[i - 1];
    while ( j > 0 && s1[j] != s1[i] )
      j = pi[j - 1];
    if ( s1[j] == s1[i] )
      ++j;
    pi[i] = j;
  }
```

```
}
```

5.2 Trie 树

```
int nxt[N][150], cnt;
int num[N];
void insert( char* s, int len ) {
  // 字符串 s[0, len)
  // 0 为根结点
  int p = 0;
  REP( i, 0, len - 1 ) {
    if ( !nxt[p][s[i]] )
      nxt[p][s[i]] = ++cnt;
    p = nxt[p][s[i]];
    ++num[p]; // 对前缀的统计在此处进行
  }
  // 对字符串整串的统计在此处进行
}

int find( char* s, int len ) {
  int p = 0;
  REP( i, 0, len - 1 ) {
    if ( !nxt[p][s[i]] )
      return 0;
    p = nxt[p][s[i]];
  }
  return num[p];
}
```

5.3 Manacher

```
// s[0, n)
// d1[i] 表示 [i-d1[i], i+d1[i]] 是奇数长度回文 (i 为中心)
// d2[i] 表示 [i-d2[i]+1, i+d2[i]] 是偶数长度回文 (i+0.5 为中心)
// d2[i] 为 0 表示不存在

int l = 0, r = -1;
for ( int i = 0; i < n; ++i ) {
  if ( r < i ) {
    int j = i;
```

```
    while ( j > 0 && 2 * i - ( j - 1 ) < n && s[j - 1] == s[2 * i - ( j - 1 )] )
      --j;
    d1[i] = i - j;
    l = j, r = 2 * i - j;
  } else {
    int j = ( l + r ) - i;
    if ( j - d1[j] <= l ) {
      int k = i * 2 - r;
      while ( k > 0 && 2 * i - ( k - 1 ) < n
              && s[k - 1] == s[2 * i - ( k - 1 )] )
        --k;
      d1[i] = i - k;
      l = k, r = 2 * i - k;
    } else
      d1[i] = d1[j];
  }
}

l = 0, r = -1;
for ( int i = 0; i + 1 < n; ++i ) {
  if ( r < i ) {
    int j = i;
    while ( j >= 0 && i + i + 1 - j < n && s[j] == s[i + i + 1 - j] )
      --j;
    d2[i] = i - j;
    l = j + 1, r = i + i + 1 - j - 1;
  } else {
    int j = ( l + r ) - i - 1;
    if ( j < 0 || j - d2[j] < l ) {
      int k = i + i + 1 - r - 1;
      while ( k >= 0 && i + i + 1 - k < n && s[k] == s[i + i + 1 - k] )
        --k;
      d2[i] = i - k;
      l = k + 1, r = i + i + 1 - k - 1;
    } else
      d2[i] = d2[j];
  }
```

```
}
```

5.4 回文自动机

```
// num 是该结点表示的前缀的回文后缀个数
// cnt 是该结点表示的回文串在原串中的出现次数（使用前需要向父亲更新）

namespace pam {

int t[N][26], fa[N], len[N], rs[N], cnt[N], num[N];
int sz, n, last;
int _new( int l ) {
  len[sz] = l;
  cnt[sz] = num[sz] = 0;
  return sz++;
}
// 记得 init
void init() {
  memset( t, 0, sz * sizeof t[0] );
  rs[n = sz = 0] = -1;
  last = _new( 0 );
  fa[last] = _new( -1 );
}
int get_fa( int x ) {
  while ( rs[n - 1 - len[x]] != rs[n] )
    x = fa[x];
  return x;
}
void ins( int ch ) {
  rs[++n] = ch;
  int p = get_fa( last );
  if ( !t[p][ch] ) {
    int np = _new( len[p] + 2 );
    num[np] = num[fa[np] = t[get_fa( fa[p] )][ch]] + 1;
    t[p][ch] = np;
  }
  ++cnt[last = t[p][ch]];
}

```

```
}
```

5.5 字符串哈希

```
#define ENABLE_DOUBLE_HASH

namespace hashstr {

typedef unsigned long long ull;
const int x = 135;
const int p1 = 0x3b3b3bbb, p2 = 1e9 + 1011;
ull xp1[N], xp2[N];

// 记得 init
void init_xp() {
  xp1[0] = xp2[0] = 1;
  for ( int i = 1; i < N; ++i ) {
    xp1[i] = xp1[i - 1] * x % p1;
    xp2[i] = xp2[i - 1] * x % p2;
  }
}

struct str {
  char s[N];
  int length;
  ull h[N];

  ull hash() {
    length = strlen( s );
    ull res1 = 0, res2 = 0;
    h[length] = 0;
    for ( int j = length - 1; j >= 0; --j ) {
#ifdef ENABLE_DOUBLE_HASH
      res1 = ( res1 * x + s[j] ) % p1;
      res2 = ( res2 * x + s[j] ) % p2;
      h[j] = ( res1 << 32 ) | res2;
#else
      res1 = ( res1 * x + s[j] ) % p1;
      h[j] = res1;
#endif
```

```
    }
    return h[0];
  }
  // 左闭右开区间
  ull get_substr_hash( int left, int right ) const {
    int len = right - left;
#ifdef ENABLE_DOUBLE_HASH
    unsigned int mask32 = ~( 0u );
    ull left1 = h[left] >> 32, right1 = h[right] >> 32;
    ull left2 = h[left] & mask32, right2 = h[right] & mask32;
    return ( ( ( left1 - right1 * xp1[len] % p1 + p1 ) % p1 ) << 32 )
           | ( ( ( left2 - right2 * xp2[len] % p2 + p2 ) % p2 ) );
#else
    return ( ( h[left] - h[right] * xp1[len] % p1 + p1 ) % p1 );
#endif
  }
};

}
```

5.6 后缀数组

```
int buf1[N], buf2[N], buc[N];

// str 存储在下标 [1, n] 的范围内，str[0] 要与 str[1..n] 都不相同
// ht[i] = LCP(sa[i], sa[i-1])
void sufsort( char str[], int n, int sa[], int rk[], int ht[] ) {
  int *x = buf1, *y = buf2, m = 127;
  REP( i, 0, m ) buc[i] = 0;
  REP( i, 1, n ) buc[x[i] = str[i]]++;
  REP( i, 1, m ) buc[i] += buc[i - 1];
  REPd( i, n, 1 ) sa[buc[x[i]]--] = i;
  for ( int k = 1; k <= n; k <<= 1 ) {
    int p = 0;
    REP( i, n - k + 1, n ) y[++p] = i;
    REP( i, 1, n ) if ( sa[i] > k ) y[++p] = sa[i] - k;
    REP( i, 0, m ) buc[i] = 0;
    REP( i, 1, n ) buc[x[y[i]]]++;
    REP( i, 1, m ) buc[i] += buc[i - 1];
```

```
    REPd( i, n, 1 ) sa[buc[x[y[i]]]--] = y[i];
    swap( x, y ), x[sa[1]] = p = 1;
    REP( i, 2, n )
    if ( y[sa[i - 1]] == y[sa[i]] && y[sa[i - 1] + k] == y[sa[i] +
        k] )
      x[sa[i]] = p;
    else
      x[sa[i]] = ++p;
    if ( ( m = p ) >= n )
      break;
  }
  REP( i, 1, n ) rk[sa[i]] = i;
  for ( int j = 0, k = 0, i = 1; i <= n; ht[rk[i++]] = k )
    for ( k ? k-- : 0, j = sa[rk[i] - 1]; str[i + k] == str[j + k];
        k++ )
      ;
}
```

6 STL

6.1 算法库

不修改序列的操作

批量操作

在标头 <algorithm> 定义

`for_each`

应用一元函数对象到范围中元素 (函数模板)

`ranges::for_each` (C++20)

应用一元函数对象到范围中元素 (算法函数对象)

`for_each_n` (C++17)

应用函数对象到序列的前 N 个元素 (函数模板)

`ranges::for_each_n` (C++20)

应用函数对象到序列的前 N 个元素 (算法函数对象)

搜索操作

在标头 <algorithm> 定义

`all_of` (C++11)

`any_of` (C++11)

`none_of` (C++11)

检查谓词是否对范围中所有、任一或无元素为 `true` (函数模板)

`ranges::all_of` (C++20)

`ranges::any_of` (C++20)
`ranges::none_of` (C++20)
检查谓词是否对范围中所有、任一或无元素为 `true` (算法函数对象)
`ranges::contains` (C++23)
`ranges::contains_subrange` (C++23)
检查范围是否包含给定元素或子范围 (算法函数对象)
`find`
`find_if`
`find_if_not` (C++11)
查找首个满足特定条件的元素 (函数模板)
`ranges::find` (C++20)
`ranges::find_if` (C++20)
`ranges::find_if_not` (C++20)
查找首个满足特定条件的元素 (算法函数对象)
`ranges::find_last` (C++23)
`ranges::find_last_if` (C++23)
`ranges::find_last_if_not` (C++23)
查找最后一个满足特定条件的元素 (算法函数对象)
`find_end`
查找元素序列在特定范围中最后一次出现 (函数模板)
`ranges::find_end` (C++20)
查找元素序列在特定范围中最后一次出现 (算法函数对象)
`find_first_of`
搜索一组元素中任一元素 (函数模板)
`ranges::find_first_of` (C++20)
搜索一组元素中任一元素 (算法函数对象)
`adjacent_find`
查找首对相同（或满足给定谓词）的相邻元素 (函数模板)
`ranges::adjacent_find` (C++20)
查找首对相同（或满足给定谓词）的相邻元素 (算法函数对象)
`count`
`count_if`
返回满足特定条件的元素数目 (函数模板)
`ranges::count` (C++20)
`ranges::count_if` (C++20)
返回满足特定条件的元素数目 (算法函数对象)
`mismatch`
查找两个范围的首个不同之处 (函数模板)
`ranges::mismatch` (C++20)
查找两个范围的首个不同之处 (算法函数对象)

`equal`

判断两组元素是否相同 (函数模板)

`ranges::equal` (C++20)

判断两组元素是否相同 (算法函数对象)

`search`

搜索元素范围的首次出现 (函数模板)

`ranges::search` (C++20)

搜索元素范围的首次出现 (算法函数对象)

`search_n`

搜索元素在范围中首次连续若干次出现 (函数模板)

`ranges::search_n` (C++20)

搜索元素在范围中首次连续若干次出现 (算法函数对象)

`ranges::starts_with` (C++23)

检查一个范围是否始于另一范围 (算法函数对象)

`ranges::ends_with` (C++23)

检查一个范围是否终于另一范围 (算法函数对象)

折叠操作 (C++23 起)

在标头 `<algorithm>` 定义

`ranges::fold_left` (C++23)

左折叠范围中元素 (算法函数对象)

`ranges::fold_left_first` (C++23)

以首元素为初值左折叠范围中元素 (算法函数对象)

`ranges::fold_right` (C++23)

右折叠范围中元素 (算法函数对象)

`ranges::fold_right_last` (C++23)

以末元素为初值右折叠范围中元素 (算法函数对象)

`ranges::fold_left_with_iter` (C++23)

左折叠范围中元素，并返回 `pair`（迭代器，值）(算法函数对象)

`ranges::fold_left_first_with_iter` (C++23)

以首元素为初值左折叠范围中元素，并返回 `pair`（迭代器，optional）(算法函数对象)

修改序列的操作

复制操作

在标头 `<algorithm>` 定义

`copy`

`copy_if` (C++11)

复制范围中元素到新位置 (函数模板)

`ranges::copy` (C++20)

`ranges::copy_if` (C++20)

复制范围中元素到新位置 (算法函数对象)

`copy_n` (C++11)

复制若干元素到新位置 (函数模板)

`ranges::copy_n` (C++20)

复制若干元素到新位置 (算法函数对象)

`copy_backward`

从后往前复制范围中元素 (函数模板)

`ranges::copy_backward` (C++20)

从后往前复制范围中元素 (算法函数对象)

`move` (C++11)

将范围中元素移到新位置 (函数模板)

`ranges::move` (C++20)

将范围中元素移到新位置 (算法函数对象)

`move_backward` (C++11)

从后往前将范围中元素移到新位置 (函数模板)

`ranges::move_backward` (C++20)

从后往前将范围中元素移到新位置 (算法函数对象)

交换操作

在标头 `<algorithm>` 定义 (C++11 前)

在标头 `<utility>` 定义 (C++11 起)

在标头 `<string_view>` 定义

`swap`

在标头 `<algorithm>` 定义

交换两个对象的值 (函数模板)

`swap_ranges`

交换两个范围的元素 (函数模板)

`ranges::swap_ranges` (C++20)

交换两个范围的元素 (算法函数对象)

`iter_swap`

交换两个迭代器所指向的元素 (函数模板)

变换操作

在标头 `<algorithm>` 定义

`transform`

应用函数到元素范围，并在目标范围存储结果 (函数模板)

`ranges::transform` (C++20)

应用函数到元素范围 (算法函数对象)

`replace`

`replace_if`

替换所有满足特定条件的值为另一个值 (函数模板)

`ranges::replace` (C++20)

`ranges::replace_if` (C++20)

替换所有满足特定条件的值为另一个值 (算法函数对象)

`replace_copy`

`replace_copy_if`

复制范围，并将满足特定条件的元素替换为另一个值 (函数模板)

`ranges::replace_copy` (C++20)

`ranges::replace_copy_if` (C++20)

复制范围，并将满足特定条件的元素替换为另一个值 (算法函数对象)

生成操作

在标头 `<algorithm>` 定义

`fill`

以复制的方式赋给定值到范围中所有元素 (函数模板)

`ranges::fill` (C++20)

赋给定值到范围中元素 (算法函数对象)

`fill_n`

以复制的方式赋给定值到范围中 N 个元素 (函数模板)

`ranges::fill_n` (C++20)

赋给定值到若干元素 (算法函数对象)

`generate`

赋连续函数调用结果到范围中所有元素 (函数模板)

`ranges::generate` (C++20)

将函数结果保存到范围中 (算法函数对象)

`generate_n`

赋连续函数调用结果到范围中 N 个元素 (函数模板)

`ranges::generate_n` (C++20)

保存 N 次函数应用的结果 (算法函数对象)

移除操作

在标头 `<algorithm>` 定义

`remove`

`remove_if`

移除满足特定条件的元素 (函数模板)

`ranges::remove` (C++20)

`ranges::remove_if` (C++20)

移除满足特定条件的元素 (算法函数对象)

`remove_copy`

`remove_copy_if`

复制范围并忽略满足特定条件的元素 (函数模板)

`ranges::remove_copy` (C++20)

`ranges::remove_copy_if` (C++20)

复制范围并忽略满足特定条件的元素 (算法函数对象)

`unique`

移除范围中连续重复元素 (函数模板)

`ranges::unique` (C++20)

移除范围中连续重复元素 (算法函数对象)

`unique_copy`

创建某范围的不含连续重复元素的副本 (函数模板)

`ranges::unique_copy` (C++20)

创建某范围的不含连续重复元素的副本 (算法函数对象)

顺序变更操作

在标头 `<algorithm>` 定义

`reverse`

逆转范围中的元素顺序 (函数模板)

`ranges::reverse` (C++20)

逆转范围中的元素顺序 (算法函数对象)

`reverse_copy`

创建范围的逆向副本 (函数模板)

`ranges::reverse_copy` (C++20)

创建范围的逆向副本 (算法函数对象)

`rotate`

旋转范围中的元素顺序 (函数模板)

`ranges::rotate` (C++20)

旋转范围中的元素顺序 (算法函数对象)

`rotate_copy`

复制并旋转元素范围 (函数模板)

`ranges::rotate_copy` (C++20)

复制并旋转元素范围 (算法函数对象)

`shift_left` (C++20)

`shift_right` (C++20)

迁移范围中元素 (函数模板)

`ranges::shift_left` (C++23)

`ranges::shift_right` (C++23)

迁移范围中元素 (算法函数对象)

`random_shuffle` (C++17 前)

`shuffle` (C++11)

随机重排范围中元素 (函数模板)

`ranges::shuffle` (C++20)

随机重排范围中元素 (算法函数对象)

采样操作

在标头 `<algorithm>` 定义

`sample` (C++17)

从序列中随机选择 N 个元素 (函数模板)

`ranges::sample` (C++20)

从序列中随机选择 N 个元素 (算法函数对象)

排序和相关操作

要求

部分算法要求由实参表示的序列“已排序”或“已划分”。未满足要求时行为未定义。

序列 `[start, finish)` 在满足以下条件时已按表达式 `f(e)` 划分：存在一个整数 `n`，使得对于 `[0, std::distance(start, finish))` 中的所有整数 `i`，`f(*(start + i))` 当且仅当 `i < n` 时是 `true`。

划分操作

在标头 `<algorithm>` 定义

`is_partitioned` (C++11)

判断范围是否已按给定谓词划分 (函数模板)

`ranges::is_partitioned` (C++20)

判断范围是否已按给定谓词划分 (算法函数对象)

`partition`

将范围中元素分为两组 (函数模板)

`ranges::partition` (C++20)

将范围中元素分为两组 (算法函数对象)

`partition_copy` (C++11)

复制范围并将元素分为两组 (函数模板)

`ranges::partition_copy` (C++20)

复制范围并将元素分为两组 (算法函数对象)

`stable_partition`

将元素分为两组，同时保留其相对顺序 (函数模板)

`ranges::stable_partition` (C++20)

将元素分为两组，同时保留其相对顺序 (算法函数对象)

`partition_point` (C++11)

定位已划分范围的划分点 (函数模板)

`ranges::partition_point` (C++20)

定位已划分范围的划分点 (算法函数对象)

排序操作

在标头 `<algorithm>` 定义

`sort`

将范围按升序排序 (函数模板)

`ranges::sort` (C++20)

将范围按升序排序 (算法函数对象)

`stable_sort`

将范围中元素排序，同时保持相等元之间的顺序 (函数模板)

`ranges::stable_sort` (C++20)

将范围中元素排序，同时保持相等元之间的顺序 (算法函数对象)

`partial_sort`

将范围中前 N 个元素排序 (函数模板)

`ranges::partial_sort` (C++20)

将范围中前 N 个元素排序 (算法函数对象)

`partial_sort_copy`

复制范围中元素并部分排序 (函数模板)

`ranges::partial_sort_copy` (C++20)

复制范围中元素并部分排序 (算法函数对象)

`is_sorted` (C++11)

检查范围是否已按升序排列 (函数模板)

`ranges::is_sorted` (C++20)

检查范围是否已按升序排列 (算法函数对象)

`is_sorted_until` (C++11)

找出最大的有序子范围 (函数模板)

`ranges::is_sorted_until` (C++20)

找出最大的有序子范围 (算法函数对象)

`nth_element`

将给定范围部分排序，确保其按给定元素划分 (函数模板)

`ranges::nth_element` (C++20)

将给定范围部分排序，确保其按给定元素划分 (算法函数对象)

二分搜索操作（在已划分范围上）

在标头 `<algorithm>` 定义

`lower_bound`

返回首个不小于给定值的元素的迭代器 (函数模板)

`ranges::lower_bound` (C++20)

返回首个不小于给定值的元素的迭代器 (算法函数对象)

`upper_bound`

返回首个大于给定值的元素的迭代器 (函数模板)

`ranges::upper_bound` (C++20)

返回首个大于给定值的元素的迭代器 (算法函数对象)

`equal_range`

返回匹配特定键值的元素范围 (函数模板)

`ranges::equal_range` (C++20)

返回匹配特定键值的元素范围 (算法函数对象)

`binary_search`

判断元素是否在偏序范围中 (函数模板)

`ranges::binary_search` (C++20)

判断元素是否在偏序范围中 (算法函数对象)

集合操作（在已排序范围上）

在标头 `<algorithm>` 定义

`includes`

当一个序列是另一个的子序列时返回 `true` (函数模板)

`ranges::includes` (C++20)

当一个序列是另一个的子序列时返回 `true` (算法函数对象)

`set_union`

计算两个集合的并集 (函数模板)

`ranges::set_union` (C++20)

计算两个集合的并集 (算法函数对象)

`set_intersection`

计算两个集合的交集 (函数模板)

`ranges::set_intersection` (C++20)

计算两个集合的交集 (算法函数对象)

`set_difference`

计算两个集合的差集 (函数模板)

`ranges::set_difference` (C++20)

计算两个集合的差集 (算法函数对象)

`set_symmetric_difference`

计算两个集合的对称差 (函数模板)

`ranges::set_symmetric_difference` (C++20)

计算两个集合的对称差 (算法函数对象)

归并操作（在已排序范围上）

在标头 `<algorithm>` 定义

`merge`

合并两个有序范围 (函数模板)

`ranges::merge` (C++20)

合并两个有序范围 (算法函数对象)

`inplace_merge`

就地合并两个有序范围 (函数模板)

`ranges::inplace_merge` (C++20)

就地合并两个有序范围 (算法函数对象)

堆操作

(C++20 前) 随机访问范围 `[first, last)` 在满足以下条件时是一个关于比较器 `comp` 的堆：对于 `(0, last - first)` 中的所有整数 `i`，`bool(comp(first[(i - 1)/ 2], first[i]))` 都是 `false`。

在标头 `<algorithm>` 定义

`push_heap`

添加元素到最大堆 (函数模板)

`ranges::push_heap` (C++20)

添加元素到最大堆 (算法函数对象)

`pop_heap`

移除最大堆中最大元 (函数模板)

`ranges::pop_heap` (C++20)

移除最大堆中最大元 (算法函数对象)

`make_heap`

从元素范围创建最大堆 (函数模板)

`ranges::make_heap` (C++20)

从元素范围创建最大堆 (算法函数对象)

`sort_heap`

将最大堆变成按升序排序的元素范围 (函数模板)

`ranges::sort_heap` (C++20)

将最大堆变成按升序排序的元素范围 (算法函数对象)

`is_heap`

检查给定范围是否为最大堆 (函数模板)

`ranges::is_heap` (C++20)

检查给定范围是否为最大堆 (算法函数对象)

`is_heap_until` (C++11)

查找能成为最大堆的最大子范围 (函数模板)

`ranges::is_heap_until` (C++20)

查找能成为最大堆的最大子范围 (算法函数对象)

最小/最大操作

在标头 `<algorithm>` 定义

`max`

返回给定值中较大者 (函数模板)

`ranges::max` (C++20)

返回给定值中较大者 (算法函数对象)

`max_element`

返回范围中最大元 (函数模板)

`ranges::max_element` (C++20)

返回范围中最大元 (算法函数对象)

`min`

返回给定值中较小者 (函数模板)

`ranges::min` (C++20)

返回给定值中较小者 (算法函数对象)

`min_element`

返回范围中最小元 (函数模板)

`ranges::min_element` (C++20)

返回范围中最小元 (算法函数对象)

`minmax` (C++11)

返回两个元素间的较小者和较大者 (函数模板)

`ranges::minmax` (C++20)

返回两个元素间的较小者和较大者 (算法函数对象)

`minmax_element` (C++11)

返回范围中的最小元和最大元 (函数模板)

`ranges::minmax_element` (C++20)

返回范围中的最小元和最大元 (算法函数对象)

`clamp` (C++17)

在一对边界值下夹逼一个值 (函数模板)

`ranges::clamp` (C++20)

在一对边界值下夹逼一个值 (算法函数对象)

字典序比较操作

在标头 `<algorithm>` 定义

`lexicographical_compare`

当一个范围字典序小于另一个时返回 `true` (函数模板)

`ranges::lexicographical_compare` (C++20)

当一个范围字典序小于另一个时返回 `true` (算法函数对象)

`lexicographical_compare_three_way` (C++20)

三路比较两个范围 (函数模板)

排列操作

在标头 `<algorithm>` 定义

`next_permutation`

生成元素范围的下一个字典序更大的排列 (函数模板)

`ranges::next_permutation` (C++20)

生成元素范围的下一个字典序更大的排列 (算法函数对象)

`prev_permutation`

生成元素范围的下一个字典序更小的排列 (函数模板)

`ranges::prev_permutation` (C++20)

生成元素范围的下一个字典序更小的排列 (算法函数对象)

`is_permutation` (C++11)

判断一个序列是否为另一个序列的排列 (函数模板)

`ranges::is_permutation` (C++20)

判断一个序列是否为另一个序列的排列 (算法函数对象)

数值运算

在标头 `<numeric>` 定义

`iota` (C++11)

从初始值开始连续递增填充范围 (函数模板)

`ranges::iota` (C++23)

从初始值开始连续递增填充范围 (算法函数对象)

`accumulate`

求和或折叠范围中元素 (函数模板)

`inner_product`

计算两个范围中元素的内积 (函数模板)

`adjacent_difference`

计算范围中相邻元素的差 (函数模板)

`partial_sum`

计算范围中元素的部分和 (函数模板)

`reduce` (C++17)

类似 `std::accumulate`，但不依序执行 (函数模板)

`exclusive_scan` (C++17)

类似 `std::partial_sum`，第 `i` 个和中排除第 `i` 个输入 (函数模板)

`inclusive_scan` (C++17)

类似 `std::partial_sum`，第 `i` 个和中包含第 `i` 个输入 (函数模板)

`transform_reduce` (C++17)

应用可调用对象，然后乱序规约 (函数模板)

`transform_exclusive_scan` (C++17)

应用可调用对象，然后计算排除扫描 (函数模板)

`transform_inclusive_scan` (C++17)

应用可调用对象，然后计算包含扫描 (函数模板)